

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবেরো

(Prof. Chester Rebeiro)

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ

(Department of Computer Science and Engineering),

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাজ

(Indian Institute of Technology, Madras)

সপ্তাহ – 07

বক্তৃতা – 31

ডেডলকস

(Deadlocks)

হ্যালো। আগের ভিডিওতে, আমরা Dining Philosopher Problem দেখেছি। এবং আমরা সমস্যাটির জন্য সমাধানগুলি দেখেছি, যাকে deadlock বলা হয়। সুতরাং, এই ভিডিওতে, আমরা deadlock সম্পর্কে আরো বিস্তারিত বিবেচনা করব, এবং কিভাবে তারা সিস্টেমের মধ্যে পরিচালিত হয় দেখব।

(স্লাইড সময় : 00:39 দেখুন)

তাই আসুন আমরা বলি আমাদের দুটি প্রসেস A এবং B আছে, এবং আমাদের দুটি resource আছে R1 এবং R2. সুতরাং, এই resource গুলি একটি সিস্টেমের কিছু হতে পারে যা সীমিত পরিমাণে থাকে। উদাহরণস্বরূপ, resource গুলি disk এর মধ্যে সংরক্ষিত file হিসাবে ছোট ছোট কিছু হতে পারে বা এটি একটি printer হতে পারে যা print করতে ব্যবহৃত হয় বা একটি plotter, scanner ইত্যাদি হতে পারে। তাই, মূলতঃ R1 থেকে A এর তীরটি নির্দেশ করে যে বর্তমানে A R1 resource ধারণ বা hold করে। উদাহরণস্বরূপ, যদি R1 ফাইল হয়; এর মানে, A বর্তমানে ফাইলটি exclusively খুলছে এবং ফাইলটিতে কিছু অপারেশন করছে। অনুরূপভাবে,

resource R2 টি B দ্বারা hold হয়। উদাহরণস্বরূপ, যদি এটি R2 printer হয়, B বর্তমানে কোন বিশেষ document print করার জন্য printer ব্যবহার করছে।

(স্লাইড সময় : 01:52 দেখুন)

এখন, এই বিশেষ পরিস্থিতিটি বিবেচনা করুন যেখানে প্রসেস A- hold resource R1 এবং প্রক্রিয়া B- র hold resource R2; কিন্তু একই সময়ে, A একটি R2 এর ব্যবহার করার অনুরোধ করছে। সুতরাং, প্রাথমিকভাবে A কে R2 প্রাপ্তির জন্য অপেক্ষা করছে; এবং প্রক্রিয়া B R1 resource পাওয়ার জন্য অপেক্ষা করছে। সুতরাং, একটি উদাহরণ নিতে, প্রক্রিয়া A disk এর মধ্যে সংরক্ষিত file খুলছে। এবং একই সময়ে, উদাহরণস্বরূপ, এটি file টি resource R2 তে print করতে চায় যা আমরা ধরেছিলাম এটি একটি printer ছিল। এখন অনুরূপভাবে, প্রক্রিয়া B বর্তমানে এই resource hold করে এই resource ব্যবহার করে এবং resource R1 ব্যবহার করতে চায়। সুতরাং, আমরা এখানে দেখি যে, এটি একটি অবস্থা যাকে deadlock বলে। মূলত, একটি deadlock সিস্টেমের একটি state যেখানে প্রতিটি প্রক্রিয়া একটি event এর অপেক্ষায় আছে যা এই সেটের অন্য একটি প্রক্রিয়া করতে পারে। উদাহরণস্বরূপ, এখানে, প্রক্রিয়া A, resource R2 এর জন্য অপেক্ষা করছে যা B hold করেছে; অপরদিকে B R1 এর জন্য অপেক্ষা করছে যেটা A hold করে আছে। আমাদের দুটি প্রসেস A এবং B এর একটি সেট আছে; এবং সেটের প্রতিটি প্রক্রিয়া অপর প্রক্রিয়ায় কিছু করার জন্য অপেক্ষা করছে। উদাহরণস্বরূপ, A B- এর জন্য অপেক্ষা করছে এই resource R2 ছেড়ে দেওয়ার জন্য যখন B R1 resource ছেড়ে দেওয়ার জন্য A এর জন্য অপেক্ষা করছে। সুতরাং, এটির মত deadlock সিস্টেমে অত্যন্ত সংকটপূর্ণ পরিস্থিতি। এবং যখন এই deadlocks ঘটবে, এটি এই ক্ষেত্রে A এবং B প্রক্রিয়া একটি অসীম সময় জন্য কোন দরকারী কাজ না করে অবিরামভাবে অপেক্ষা করে। সুতরাং, এই ধরনের deadlocks পুঙ্খানুপুঙ্খ বিশ্লেষণ করা উচিত। সুতরাং, এই বিশেষ ভিডিওতে, আমরা দেখব কিভাবে সিস্টেমগুলির মধ্যে এই deadlock গুলি পরিচালনা করা হয়। এখন, deadlock গুলি বিচার করার জন্য, আমরা এই graph গুলি ব্যবহার করি, এইগুলিকে Resource Allocation Graph বলা হয়।

(স্লাইড সময় : 04:55 দেখুন)

এখন resource allocation graph হল সিস্টেমের বিভিন্ন resource বরাদ্দকরণের জন্য ব্যবহৃত directed গ্রাফ। এবং সেখানে deadlock ঘটেছে বা deadlock সম্ভাব্য ঘটতে যাচ্ছে কিনা তা নির্ধারণ করে। সুতরাং, এই directed graph এ, বর্গক্ষেত্র দ্বারা resource represent করা হয়। সুতরাং, উদাহরণে R1 এবং R2 resource এবং তারা বর্গক্ষেত্র দ্বারা represent করা হয়। অনুরূপভাবে, এখানে প্রদর্শিত হিসাবে বৃত্তগুলি প্রসেস represent করার জন্য ব্যবহৃত হয়।

(স্লাইড সময় : 05:37 দেখুন)

এবং আগে আমরা দেখেছি যে, প্রক্রিয়া থেকে resource এর দিকে তীরটি নির্দেশ করে যে এই resource এর জন্য একটি অনুরোধ করা হয়েছে। উদাহরণস্বরূপ, এখানে A থেকে R1 এর তীরটি ইঙ্গিত দেয় যে A resource R1 অনুরোধ করে; অনুরূপভাবে B, এই ক্ষেত্রে resource R2 এর জন্য অনুরোধ করেছে। সুতরাং, এই অনুরোধ অপারেটিং সিস্টেমকে করা হয় এবং যদি সম্ভব হয় অপারেটিং সিস্টেম তারপর সংশ্লিষ্ট resource বরাদ্দ করবে।

(স্লাইড সময় : 06:22 দেখুন)

সুতরাং, যখন graph টি এইরকম হবে, মূলত তীরের দিকটি পরিবর্তিত হয়েছে এখন তীরটি R1 থেকে A এর দিকে চলে যায়, অর্থাৎ A প্রসেস R1 resource hold করে। একইভাবে, R2 থেকে B এর তীর নির্দেশ করে যে B hold করে resource R2। Deadlock এর জন্য চারটি শর্ত আছে। সুতরাং, আমরা এখন একটি deadlock এর জন্য এই শর্তগুলি দেখব।

(স্লাইড সময় : 06:59 দেখুন)

সুতরাং, প্রথমটি mutual exclusion। সুতরাং, আমরা এই দ্বারা বোঝানো হয় যে সিস্টেমের প্রতিটি resource হয় available বা বর্তমানে ঠিক একটি প্রক্রিয়াকে নিযুক্ত বা assign করে। সুতরাং, উদাহরণস্বরূপ এখানে আমাদের resource R1 আছে যা available। সুতরাং, এটি কোনো নির্দিষ্ট প্রক্রিয়া assign করে না, ঠিক আছে। এও ঠিক আছে যেখানে resource টি একটি প্রক্রিয়াতে assigned করা হয়, কিন্তু deadlock ঘটার জন্য এই ধরনের অবস্থা উচিত অর্থাৎ resource টি দুটি প্রসেস A এবং B এর মধ্যে share করা যাবে না।

(স্লাইড সময় : 07: 48 দেখুন)

একটি deadlock এর জন্য পরবর্তী শর্ত hold and wait করা হয় যে একটি প্রক্রিয়া একটি resource hold করে অন্য resource অনুরোধ করতে পারে, এই ক্ষেত্রে A প্রক্রিয়া দ্বারা resource R1 hold হয় এবং R1 থাকার সময়, A এছাড়াও R2 অনুরোধ করে, A মূলত R 1 hold করে এবং R2 এর জন্য অপেক্ষা করেছে।

(স্লাইড সময় : 08: 22 দেখুন)

একটি deadlock ঘটার তৃতীয় শর্ত হল No preemption। মূলত, এটি এমন কোনও ক্ষেত্রে না হওয়া উচিত যা কোনও নির্দিষ্ট প্রক্রিয়ার জন্য assigned resource অপারেটিং সিস্টেম সেই প্রক্রিয়া থেকে জোর করে নেওয়া হয় না। অর্থাৎ অপারেটিং সিস্টেম বা অন্য কেউ একটি নির্দিষ্ট প্রক্রিয়াকে বরাদ্দ resource জোরপূর্বক অপসারণ করতে পারে না। পরিবর্তে প্রসেসগুলি স্পষ্টভাবে নিজের দ্বারা release করা উচিত যে যখনই প্রক্রিয়াটি চায় সেটি নিজেই resource মুক্ত করবে।

(স্লাইড সময় : 09:18 দেখুন)

সুতরাং, চতুর্থ প্রয়োজনটি হল Circular wait। এর অর্থ এই যে, দুই বা তার বেশি প্রসেসের একটি circular chain রয়েছে, যা প্রতিটি chain এর পরবর্তী সদস্য দ্বারা held একটি resource এর জন্য অপেক্ষা করছে। সুতরাং, আমরা এখানে দেখি যে আমাদের একটি circular chain আছে এবং এখানে একটি অপেক্ষা আছে কারণ প্রক্রিয়া A অপেক্ষা করছে B প্রক্রিয়ার জন্য R2 release করার জন্য; এবং প্রক্রিয়া B প্রক্রিয়া A- এর R1 release করার জন্য অপেক্ষা করছে। সুতরাং, এখানে একটি circular wait condition আছে।

(স্লাইড সময় : 10:00 দেখুন)

সুতরাং, এই 4 টি শর্তাবলী: mutual exclusion, hold and wait, no preemption এবং circular wait কোন সিস্টেমের মধ্যে উপস্থিত হওয়া আবশ্যিক যাতে একটি deadlock ঘটতে পারে। সুতরাং, উদাহরণস্বরূপ, আমরা একটি সিস্টেম নির্মাণ করতে পারি যেখানে এইগুলির কোন একটি উপস্থিত নেই। উদাহরণস্বরূপ, ধরুন আমরা এমন একটি সিস্টেম তৈরি করি যেখানে প্রসেসগুলি কোনো নির্দিষ্ট সম্পদ hold রেখে এবং একই সময়ে আরেকটি resource এর জন্য অপেক্ষা করতে পারে না। সুতরাং, এই ধরনের সিস্টেমের কোন deadlock থাকবে না। অন্যদিকে, অনুমান করুন একটি সিস্টেম উন্নত করা হয়েছে যেখানে এই সমস্ত জিনিসগুলি সম্ভাব্য যে resource ব্যবহার করার সময় একটি mutual exclusion আছে, একটি প্রক্রিয়া একটি সম্পদ hold করে এবং অন্য একটির জন্য অপেক্ষা করতে পারে, একবার assigned resource জোরালোভাবে তা থেকে preempt করা যাবে না এবং circular wait অনুমতি দেওয়া হয় তাহলে deadlocks ঘটতে পারে। সুতরাং, এই সব শর্ত থাকলেই একটি deadlock ঘটেছে বোঝায় না। এটা শুধুমাত্র ভবিষ্যতে deadlock ঘটার একটি সম্ভাবনা আছে বোঝায়।

(স্লাইড সময় : 11:33 দেখুন)

সুতরাং, বলা যায় "একটি সিস্টেমের মধ্যে একটি deadlock একটি সম্ভাব্য ঘটনা" বলা হচ্ছে মূলত, এটি বিভিন্ন প্রকারের উপর নির্ভর করে যেমন পদ্ধতির দ্বারা প্রক্রিয়াগুলি অনুরোধ করা হয়, অপারেটিং সিস্টেমের দ্বারা এই resource এর জন্য allocation এবং de-allocation করার উপায় ইত্যাদি। সুতরাং, শুধুমাত্র যদি এই অনুরোধের একটি নির্দিষ্ট ক্রম ঘটে এবং শুধুমাত্র তখন একটি deadlock ঘটবে। সুতরাং, request এবং allocation এর একটি ছোট পরিবর্তন ঘটালে deadlock নাও হতে পারে। সুতরাং আসুন আমরা এইগুলির কিছু উদাহরণ দেখি। সুতরাং আসুন আমরা বলি আমাদের সিস্টেমে A, B এবং C তিনটি প্রসেস আছে, এবং তিনটি resource R, S এবং T আছে। সুতরাং, প্রতিটি প্রক্রিয়া তার execution এর কোন এক সময় resource request বা release করতে পারে। তাই স্পষ্টত, একটি request অনুরোধের পরেই release অনুরোধ করা সম্ভব। সুতরাং, এই request এবং resource release বিভিন্ন সময় উদাহরণে দেওয়া হয়, এটি নির্ভর করে কিভাবে A, B এবং C নির্ধারিত হয়

এবং কিভাবে তা কার্যকর করা হয়। সুতরাং আসুন আমরা এই বিশেষ ক্রমটি বিবেচনা করি যে A অনুরোধ করে R, এবং তারপর B অনুরোধ করে S, C অনুরোধ করে T। তারপর A অনুরোধ করে S, B অনুরোধ করে T এবং C অনুরোধ করে R। সুতরাং, এটি অনুরোধের জন্য একটি সম্ভাব্য ক্রম। সুতরাং, আমরা resource allocation graph ব্যবহার করতে পারি। সুতরাং, আমরা দেখব যে A একটি অনুরোধ করে R এর জন্য এবং অপারেটিং সিস্টেম তারপর resource R, A কে assign করা হয়, তারপর B অনুরোধ করে S এর জন্য অনুরোধ, S B কে assign হবে, তারপর C অনুরোধ করে T এর জন্য আর OS T কে C কে assign করবে। তারপর A এস কে request করবে তাই এই মত একটি লাইন আছে। B অনুরোধ করে T কে এখানে একটি লাইন আছে এবং C অনুরোধ করে R। সুতরাং, আমরা আমাদের পূর্ববর্তী স্লাইড দেখেছি যে চারটি শর্ত তার সব পূরণ করা হয়েছে। উদাহরণস্বরূপ, আপনি দেখতে circular wait এখানে পাওয়া যায়। A hold করে resource R আর A অনুরোধ করেছে S; B hold করে আছে S এবং একই সময়ে B অনুরোধ করে T। C hold করে আছে T, একই সময়ে C, R এর জন্য অনুরোধ করে। সুতরাং, আপনি দেখতে পান যে প্রতিটি প্রক্রিয়াকে একটি প্রক্রিয়া একটি নির্দিষ্ট resource release করার জন্য অপেক্ষা করছে। সুতরাং, এখানে একটি deadlocked situation।

(স্লাইড সময় : 14:55 দেখুন)

এখন, আমরা দেখতে পাব যে যদি একই request এবং allocation গুলি সামান্য ভিন্নভাবে সম্পন্ন করা হয় তবে deadlock ঘটেবে না। উদাহরণস্বরূপ, R কে বরাদ্দ করা হয় A তে এবং তারপর T কে বরাদ্দ করা হয় C তে ; তারপর A অনুরোধ করে S এবং allocation পায়। তারপর C অনুরোধ করে R, তাই আমরা এখানে আছি। এখন, A release করে R কে, তাই এখানে কোনও রেখা নেই বা এখানে edge নেই, তারপর A release করে S কে R কে বরাদ্দ করা যেতে পারে C কে। সুতরাং, আপনি দেখতে পারেন যে request গুলি এবং release এর উপর নির্ভর করে আমরা অর্জন করতে সক্ষম একটি পরিস্থিতি যেখানে প্রতিটি request এবং release অপারেটিং সিস্টেম দ্বারা পরিসেবা হতে পারে। সুতরাং, এই পরিস্থিতিতে, deadlock পাওয়া যায়নি।

(স্লাইড সময় : 16:07 দেখুন)

এখন, আমরা দেখেছি যে deadlock আসলে একটি সম্ভাব্য ঘটনা এবং কিছু probability এর সঙ্গে ঘটতে পারে। এখন সিস্টেমের মধ্যে একাধিক resource থাকলে এই probability কমতে পারে। সুতরাং, আমরা দেখেছি যে এটি একটি deadlocked state মূলত কারণ A অপেক্ষা করছে B দ্বারা R2 release হওয়ার জন্য এবং অপরদিকে B, R1 এর জন্য অপেক্ষা করছে। এখন এটা সমাধানের একটি উপায় হল একাধিক resource থাকা, এই বিশেষ সমাধান তাই সবসময় কাজ করবে না এবং মূলত resource গুলির উপর নির্ভর করে, এটি কিছুটা সাহায্য করতে পারে। উদাহরণস্বরূপ, যদি আমাদের ঠিক একই ধরনের দুটি resource থাকে তাহলে B এর অনুরোধের সাথে A এর অনুরোধ উভয়ই পরিচালিত হতে পারে, যদি resource R1 দুটি থাকে resource R 1 A এবং B উভয়কেই

দেওয়া যেতে পারে। অনুরূপভাবে, resource R 2 এক জোড়া থাকলে B এবং C কে একযোগে দেওয়া যেতে পারে। সুতরাং, এর অর্থ কি, উদাহরণস্বরূপ, আমাদের উপস্থিত দুটি printer থাকতে পারে এবং A কে একটি প্রিন্টার বরাদ্দ করা যেতে পারে, যখন B কে অন্য printer বরাদ্দ করা যেতে পারে, যখন এটি সম্পূর্ণরূপে deadlocks নির্মূল না করলেও এটি deadlock এর সম্ভাবনা কমায়।

(স্লাইড সময় : 18:00 দেখুন)

এখন, পরবর্তী প্রশ্ন একটি সিস্টেমে deadlocks থাকলে, সেই deadlock এর মোকাবিলা করা কি উচিত? সুতরাং, এটি একটি বিতর্কিত প্রশ্ন, মূলত এটি সহজে উত্তর দেওয়া যায় না, কারণ প্রতিরোধের প্রক্রিয়া (prevention mechanism) বা deadlocks সনাক্ত করার খরচ অত্যন্ত উচ্চ হয়, এবং এটি অপারেটিং সিস্টেমের মধ্যে প্রচুর overhead খরচ হবে। তাই, অন্য দিকটি ostrich algorithm হিসাবে পরিচিত যে deadlock সম্পূর্ণ উপেক্ষা করে কোনও deadlock prevention বা কোনও deadlock সনাক্তকরণ প্রক্রিয়া ছাড়া সিস্টেমটি চালানো। তাই, বিকল্প হল যে কিছু deadlock prevention বা detection প্রক্রিয়া থাকা বা deadlock সম্পূর্ণ উপেক্ষা করেই OS design এর সময় বা সিস্টেম design করা। সুতরাং, একটি সিদ্ধান্ত নেওয়া যেতে পারে আগে বিভিন্ন বিষয় আলোচনা করা প্রয়োজন, যেমন deadlock এর সম্ভাবনা কি? এটা কি সম্ভবত deadlock প্রতি সপ্তাহে, বা প্রতি মাসে বা 5 বছরে একবার ঘটবে? দ্বিতীয়ত, deadlock এর পরিণতি কি? মূলত, deadlock সংকটপূর্ণ হতে পারে? উদাহরণস্বরূপ, যদি আমার desktop এ কোনও deadlock হয়, আমি সিস্টেম reboot করতে পারি এবং এটি আমাকে বেশি প্রভাবিত করবেনা। অন্যদিকে যদি safety critical application এ একটি deadlock ঘটে, যেমন একটি মহাকাশযান বা একটি রকেট ধরনের scenario তে, তাহলে তা বিপর্যয়কর হতে পারে। অতএব, আমাদের এই দুটি দিক সম্পর্কে তর্ক করতে হবে, মূলত যদি একটি deadlock ঘটতে পারে এমন সম্ভাবনা খুব ঘন ঘন হয় তাহলে সম্ভবত deadlock মোকাবিলা করার জন্য OS এর কিছু পদক্ষেপের প্রয়োজন হবে। অন্য দিকে, যদি deadlock খুব বিক্ষিপ্ত হয় যেমন গড়ে পাঁচ বছরে একবার তাহলে অপারেটিং সিস্টেমের মধ্যে একটি deadlock handle নাও করতে পারেন। তাই এখন আমাদের অনুমান করা যাক যে আমরা আমাদের অপারেটিং সিস্টেমের কিছু ব্যবস্থার প্রয়োজন যাতে deadlock handle করা যায়। সুতরাং, আমরা এই সম্পর্কে কি করতে পারি?

(স্লাইড সময় : 20:47 দেখুন)

মূলত, তিনটি পথ রয়েছে যেভাবে deadlock handle করা যায়। এক সনাক্তকরণ এবং পুনরুদ্ধার (detection and recovery) দ্বারা, দ্বিতীয় এড়ানো (prevention) দ্বারা এবং তৃতীয় প্রতিরোধ (prevention) দ্বারা।

(স্লাইড সময় : 21:06 দেখুন)

সুতরাং আসুন আমরা প্রথম কেসটি দেখি, যা detection and recovery এবং প্রথমে কীভাবে deadlock সনাক্ত করা হয়? অপারেটিং সিস্টেমের জন্য deadlock সনাক্ত করতে মূলত, এটি সিস্টেমের বর্তমান resource allocation জানা প্রয়োজন। মূলত কোন প্রক্রিয়াটি কোন resource hold করে এবং প্রসেস দ্বারা বর্তমান resource allocation জানা প্রয়োজন হয়। মূলত কোন প্রক্রিয়াটি কোন resource এর জন্য অপেক্ষা করছে, এবং OS তারপর deadlock সনাক্ত করতে এই তথ্যটি ব্যবহার করবে।

(স্লাইড সময় : 21:46 দেখুন)

সুতরাং, সনাক্তকরণটি যেভাবে কাজ করতে পারে তা হল Resource Allocation Graph এ cycle খুঁজে পাওয়া। উদাহরণস্বরূপ, যদি এই সিস্টেমের বিভিন্ন resource গুলির জন্য resource allocation graph গ্রাফ হয়, তাহলে OS একটি cycle এর উপস্থিতি সনাক্ত করবে। উদাহরণস্বরূপ, এখানে, আমরা প্রসেস D, E এবং G এর মধ্যে একটি cycle আছে তারপর OS বলবে যে এই তিনটি প্রসেস deadlock state এ রয়েছে।

(স্লাইড সময় : 22:19 দেখুন)

এখন অন্য দিকটি এখানে দেখানো হয়েছে, যেমনটি এখানে দেখানো হয়েছে, যেখানে resource S রয়েছে এবং এটি A, D, F এবং C দ্বারা একযোগে অনুরোধ করা হয়। সুতরাং আমরা দেখতে পাচ্ছি এটি একটি deadlock অবস্থা নয় কারণ এই সমস্ত প্রসেসের জন্য S এর allocation কিছু ক্রম আছে। উদাহরণস্বরূপ, একটি সম্ভাব্য allocation হয় যে প্রথমে S কে allocate করা হয় তারপর A কিছু সময়ের জন্য resource S ব্যবহার করবে, এবং তারপর A resource S ব্যবহারটি সম্পন্ন হওয়ার পরে, S তারপর C এ allocate করা যেতে পারে এবং S এর পরের allocation বরাদ্দ করা হয় F এবং তারপর D। সুতরাং, মূলত S এর বরাদ্দকরণ এই চারটি প্রসেসের মধ্যে sequential হতে পারে। সুতরাং, এটি একটি deadlock হবে না।

(স্লাইড সময় : 23:18 দেখুন)

তবে, resource allocation graph এ একটি cycle এর উপস্থিতি সিস্টেমের মধ্যে একটি deadlock রয়েছে তা নির্দেশ করবে।

(স্লাইড সময় : 23:28 দেখুন)

Resource Allocation Graph এ cycle খোঁজার এই পদ্ধতিটি সিস্টেমের সাথে ভালভাবে কাজ করবে যেখানে প্রতিটি প্রকারের একটি resource থাকবে। এখন ধরুন আমাদের সিস্টেমে প্রতিটি ধরনের একাধিক resource আছে তাহলে অন্য অ্যালগরিদম প্রয়োজন হবে।

(স্লাইড সময় : 23:45 দেখুন)

সুতরাং আসুন আমরা একটি উদাহরণ দেখি কিভাবে এটি কাজ করবে। আসুন আমরা একটি সিস্টেমের কথা বলি যা আমাদের চারটি resource আছে: tape drive, plotter, scanner এবং CDROM। আরও 4 টি tape drive, 2 টি plotter, 3 টি scanner এবং 1 টি CDROM রয়েছে। তারপর আমাদের এটাও বলা যাক যে আমাদের সিস্টেমে P1, P2, P3 তিনটি প্রক্রিয়া আছে এবং এটি বর্তমান allocation matrix। সুতরাং, row 1 P1 প্রক্রিয়ার সাথে সম্পর্কযুক্ত। এর মানে হল যে P1-তে কোনও tape drive নেই, তার কোনও plotter নেই, 1 টি scanner allocated এবং কোনও CD-ROM নেই। অনুরূপভাবে, P2 এর 2 টি tape drive এবং 1 CDROM রম এবং P3 প্রসেসকে allocate করা হয় 1 টি plotter এবং 2 টি scanner। এখন, এই নির্দিষ্ট সেট A উপলব্ধ (available) resource সম্পদ নির্দেশ করে; মূলত 4 টি tape drive এর মধ্যে, যদি আপনি বর্তমান available matrix এ এই সংশ্লিষ্ট column টি দেখে থাকেন তবে আমরা দেখেছি যে 2 ব্যবহৃত হয়েছে। সুতরাং, 4 বিয়োগ 2 বাকি 2 টি tape drive আছে যা allocate করার জন্য উপলব্ধ আছে। সুতরাং, অনুরূপভাবে যদি আপনি plotters দেখেন 2 টি plotter এর মধ্যে একটি allocate করা হয় এবং 1 টি available। 3 টি scanner এর মধ্যে, আমরা দেখতে পাই যে সমস্ত 3 টিই allocate করা হয়েছে, তাই এখানে available 0। একইভাবে, কোনও CDROM ও বরাদ্দ করার জন্য উপলব্ধ নেই। এখন, এই ছাড়াও আমাদের একটি request matrix আছে, মূলত যার জন্য প্রক্রিয়া অপেক্ষা করছে। উদাহরণস্বরূপ, প্রসেস P1 এর request আরও 2 টি tape drive এবং 1 টি CDROM। সুতরাং, প্রসেস P2 এর জন্য 1 টি tape drive এবং 1 টি scanner প্রয়োজন, এবং প্রসেস P3 এর জন্য 2 টি tape drive ড্রাইভ এবং 1 টি plotter প্রয়োজন। এখন request allocation এবং request এর এই ধরনের উপস্থাপনা থাকার লক্ষ্য এই যে এমন কোন request sequence আছে কি না তা নির্ধারণ করা হয়, যাতে সব request পূরণ করা যেতে পারে। যদি এইরকম একটি sequence সম্ভব হয়, তাহলে কোনও deadlock উপস্থিত নেই। তাই আসুন 3 টি প্রসেসের মাধ্যমে আমরা এই request allocate করতে পারি কিনা তা দেখি।

(স্লাইড সময় : 26:52 দেখুন)

তাই আসুন আমরা একটি প্রক্রিয়া P1 নিয়ে দেখি P1 দ্বারা request পূরণ করা যেতে পারে কি না। সুতরাং, এটির 2 টি tape drive প্রয়োজন এবং আমরা দেখছি যে 2 টি tape drive উপলব্ধ। সুতরাং, এটি ঠিক আছে এবং 1 টি CDROM অনুরোধ করা হয়, কিন্তু কোনও CD-ROM উপলব্ধ নেই। সুতরাং, প্রক্রিয়া P1 চালানো যাবে না। সুতরাং, প্রক্রিয়া P1 কে তার সমস্ত resource allocate করা যাবে না। সুতরাং, এটি execute করতে পারে না।

(স্লাইড সময় : 27:17 দেখুন)

চলুন দেখি প্রক্রিয়া P 2 এর জন্য। সুতরাং, P2 এর 1 টি tape drive প্রয়োজন যা এটিতে allocate করা যেতে পারে কারণ এটি উপলব্ধ; এবং এটির 1টি scanner প্রয়োজন, কিন্তু 0 উপলব্ধ। সুতরাং, অনুরূপভাবে, প্রসেস P2 ও সন্তুষ্ট হতে পারে না, এবং P2 কে অপেক্ষা করতে হবে। এখন তৃতীয় কেসটি দেখতে দিন যেখানে P3 প্রক্রিয়াটি আছে, সেখানে 2 টি tape drive রয়েছে, যা পূরণ করা যায়, 1 টি plotter যা পূরণ করা যায় কিন্তু 1 টি request scanner যা পূরণ করা যাবে না। সুতরাং, প্রক্রিয়া P3 এর request কেও সন্তুষ্ট করা যাবে না। সুতরাং, P3 ও অপেক্ষা করবে। সুতরাং, P1, P2 এবং P3 সব request এর জন্য অপেক্ষা করছে, এবং সেইজন্য state টি একটি deadlock কারণ এই প্রক্রিয়ার যে কোনও request পূরণ করা যায় না এবং সেইজন্য, সকল প্রক্রিয়াকেই অপেক্ষা করতে হবে।

(স্লাইড সময় : 28:34 দেখুন)

যদি আপনি অন্য একটি উদাহরণ দেখেন যেখানে একটি ছোট পরিবর্তন আছে। আমরা শুধু প্রক্রিয়া 3 দ্বারা প্রয়োজনীয় scanner সংখ্যা হ্রাস করেছি। সুতরাং, এই ক্ষেত্রে, আমরা দেখতে পাই যে P3 প্রক্রিয়া দ্বারা request করা tape drive গুলি সিস্টেম দ্বারা পূরণ করা যায়, একটি plotter পূরণ করা যায় এবং কোন scanner বা কোন CD-ROM প্রয়োজন হয় না এবং তাই, আমরা দেখতে পাই যে P3 প্রক্রিয়ার জন্য সমস্ত request allocate করা যেতে পারে, সুতরাং P3 সন্তুষ্ট হতে পারে এবং আমাদের এখানে একটি deadlock নেই। সুতরাং, এই ভাবে, বর্তমান allocation matrix এবং request matrix এর উপর ভিত্তি করে অপারেটিং সিস্টেম দ্বারা deadlock সনাক্ত করা যায়।

(স্লাইড সময় : 29:32 দেখুন)

সুতরাং, একবার deadlock সনাক্ত করা হলে অপারেটিং সিস্টেমটি পরবর্তীতে কি করবে? সুতরাং, OS অনেক কিছু করতে পারে। সুতরাং, একটি জিনিস হল এটি একটি alarm দিতে পারে, যেটা user এবং administrator কে বলে যে deadlock সনাক্ত করা হয়েছে।

(স্লাইড সময় : 29:51 দেখুন)

তারপর একটি দ্বিতীয় উপায় হল preemption একটি প্রক্রিয়া থেকে resource কে বলপূর্বক নিয়ে অন্য একটি প্রক্রিয়ায় দেওয়া হবে। উদাহরণস্বরূপ, এটি R2 একটি printer এবং এটি বর্তমানে B দ্বারা held। সুতরাং, কি করা যেতে পারে তা হল printer টি B থেকে সরিয়ে নিতে বাধ্য করা যেতে পারে, যখন এটি কিছু সময়ের জন্য A কে বরাদ্দ করা হয় এবং এইভাবে deadlock ভেঙে যাবে, যেমন এখানে দেখানো হয়েছে। সুতরাং, B এর R2 resource নেই, R2 A কে দেওয়া হয়েছে।

(স্লাইড সময় : 30:42 দেখুন)

তারপর তৃতীয় পদ্ধতি হল যা rollback নামে পরিচিত একটি কৌশল ব্যবহার করে। সুতরাং, rollback এ উভয় প্রসেস A এবং B এর execution এর check point করা হবে। সুতরাং, check point মানে আমরা এই প্রক্রিয়ার state disk এ সংরক্ষণ করি। প্রক্রিয়া কিছু সময়ের জন্য execute হয় এবং তারপর প্রক্রিয়া সম্পূর্ণ state ডিস্কের মধ্যে সংরক্ষিত হয়। এখন ডিস্কে প্রসেসের অবস্থা সংরক্ষণ করে, প্রসেসগুলি এমন বিন্দু থেকে আবার চালানো যাবে যেখানে এটি check point করা হয়েছে, এটি check point state। তাই, সময়ের সাথে সাথে আরও বেশি সংখ্যক check point গুলি সময়মত এভাবে দেখানো হয়। এখন, আসুন আমরা বলি যে কোনও সময়ে একটি deadlock সনাক্ত করা গেলে কি ঘটতে পারে তা হল যে সিস্টেমটি এখানে শেষ non-deadlock state এ rollback করতে পারে। সুতরাং, rollback কীভাবে এই বিশেষ উদাহরণে A এবং B এর অবস্থাটি লোড করে দেওয়া হবে, যেমনটি এখানে প্রদর্শিত হয়েছে। এখন A এবং B প্রক্রিয়া এই অবস্থা থেকে চালিয়ে যাবে এবং deadlock আবার নাও ঘটতে পারে। মূলত আমরা দেখেছি যে যেহেতু deadlock একটি সম্ভাব্য ঘটনা যা allocation ক্রম পরিবর্তন করার মাধ্যমে আমরা এই নির্দিষ্ট deadlock প্রতিরোধ করতে পারি। একটি চতুর্থ উপায় প্রসেস গুলি kill করা। মূলত, যদি প্রক্রিয়া A এবং B একটি deadlock অবস্থায় থাকে, তাহলে এইগুলির মধ্যে একটি kill করলে deadlock ভেঙে যাবে। সুতরাং, সাধারণত কম গুরুত্বপূর্ণ বা কম priority প্রক্রিয়া kill করা হবে। ধন্যবাদ।